The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

18

Managing Orders, Customers and Refunds

Part V — Running and Extending the Shop

Taught in	Week 13
Laboratory	A refund through the order state machine
Assessment	**Assignment 2 set**

This is a class extract. It is *Chapter 18* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 18

Managing Orders, Customers and Refunds

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** how the admin panel changes an order, and **say** why it does not do so directly.
- **Predict** which status changes move stock, from the rule rather than from the status names.
- **Distinguish** the two things the platform calls a refund, and **state** what each one actually does.
- **Identify** the work a refund requires that no screen performs.
- **Design** a back-office procedure that keeps money, stock and records in agreement.

Before You Start

Chapter 10: order statuses, the history table, and status 0. Chapter 17: that the dashboard's figures are filtered differently. Chapter 14: reconciliation, and that the shop's record and the provider's must be made to agree.

Key Terms in This Chapter

back office • order history • status transition • stock subtraction • return • return action • refund • store credit • customer transaction • API signature • allow-list • audit trail • reconciliation

18.1 The Admin Panel Does Not Edit Orders

The first surprise of the back office is architectural. When an administrator changes an order's status, the admin panel does not write to the database. It **signs a request and calls the storefront's own API over HTTP**, and the storefront makes the change.

Why that is the right design

Three things must happen when an order's status changes: stock must move, the customer may need an email, and a history row must be written. Those rules are not simple, and they must be identical whether the change comes from a human clicking a dropdown, from a payment provider's callback (Chapter 14), or from a script.

Writing them once, in the storefront, and having everything else call in is the only arrangement in which they cannot drift apart. The alternative — the admin panel having its own copy — is how a shop ends up with orders whose stock was never adjusted because they were edited from the wrong screen.

The consequence nobody mentions: the admin panel cannot change an order unless the API accepts the call. The back office is a client of the shop, and it can be locked out of it.

Measured: Out of the box, the API refuses every call

Produced by running `m18_orders.py -api` in `code/`. The request is signed exactly as the startup controller verifies it: an HMAC-SHA1 over the route, the call, the username, the host, the store, the language, the currency, an MD5 of the request body and a timestamp.

Rows in `oc_api_ip`	**0**
A correctly signed call	`HTTP 403 -- You do not have permission`
After adding one address	accepted

The API user ships enabled, and its address allow-list ships empty. An empty allow-list permits nothing — the check is "is the caller's address in this list", and the answer for an empty list is always no. So every signed call is refused, including the admin panel's own.

This is the third fault of this shape in the course, after Chapter 10's missing setting and Chapter 16's unread `.htaccess`: a feature that is present, configured, enabled, and inert. In each case the system answered, politely, and did nothing.

18.2 Stock Moves on Transitions, Not on Statuses

The rule that governs stock is not written on any screen, and it is not what the status names suggest.

The rule, from `catalog/model/checkout/order.php`

The statuses are partitioned into a set — the union of `config_processing_status` and `config_complete_status` — and everything else.

Moving *into* that set subtracts stock. Moving *out of* it adds stock back. Moving within it does nothing.

In this shop the set is $\{1, 2, 3, 5, 12\}$ — Pending, Processing, Shipped, Complete and *Reversed*. Everything else, including 0, Canceled, Denied, Failed and *Refunded*, is outside it.

Measured: One order, seven status changes, and the stock after each

From `code/m18_orders.py -stock`: one order for one unit of product 43 and two of product 40, driven through the API.

Status	Where it sits	Stock 43	Stock 40
(before the order)	—	904	920
1 Pending	in the set	903	918
3 Shipped	still in the set	903	918
7 Canceled	**outside**	**904**	**920**
1 Pending	back inside	903	918
11 Refunded	**outside**	**904**	**920**
1 Pending	back inside	903	918
12 Reversed	**in the set**	**903**	**918**

Shipped changed nothing, although it is the status at which the goods physically leave. The stock left when the order was paid for.

Refunded and Reversed behave oppositely. Both names mean money went back to the customer. Status 11 returns the goods to stock; status 12 does not, because somebody put 12 in `config_processing_status`. Nothing on any screen says so, and the two statuses sit next to each other in the dropdown.

This is a setting, so it is your problem. Whoever configures a shop decides which statuses are "processing", and in doing so decides the stock behaviour of every status change the shop will ever make. It is one of the highest-consequence settings in the admin panel and it looks like a list of checkboxes.

18.3 Two Different Things Are Called a Refund

⚠ A return marked "Refunded" does nothing at all

The platform has a returns system: **Sales › Returns**, with its own statuses (Pending, Awaiting Products, Complete), five reasons, and four actions — **Waiting for return**, **Credit Issued**, **Replacement Sent** and **Refunded**.

Reading `admin/model/sale/returns.php`, recording a return performs exactly two statements:

- `UPDATE oc_return SET return_status_id = ...`
- `INSERT INTO oc_return_history ...`

And nothing else. It does not change the order's status. It does not restore stock. It does not create a customer transaction or any store credit. It does not move a single rupee.

A return marked "Refunded" is a note. It records that somebody decided to refund. The money must be sent back by hand through the payment provider, the stock adjusted by hand, and the order's own status changed by hand — and if none of that is done, the order still reads **Complete** to everyone who looks at it, including the dashboard and every report.

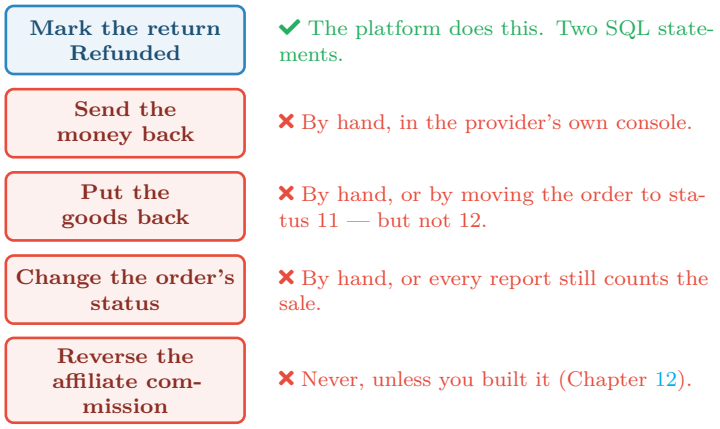


Figure 18.1: What a refund actually requires, and the one part of it the software performs. Four of the five are somebody's responsibility, and in most small shops nobody has been told which.

The procedure a shop actually needs

Because the software performs one fifth of a refund, the other four fifths have to be a written procedure, in this order:

1. **Send the money back first**, through the provider, and record the provider's reference.
2. **Move the order to status 11**, not 12, which returns the goods to stock — and only if the goods actually came back.
3. **Record the return** with the provider's reference in the comment, so that the shop's record and the provider's can be matched later (Chapter 14).
4. **Reverse any commission**, which nothing will do for you.
5. **Check the dashboard agrees** the next morning. The gap between what the shop thinks it sold and what the provider says it collected is the only thing that catches a step somebody forgot.

Steps 2 and 3 are in that order on purpose. A return recorded before the order is moved leaves a window in which the shop's records say "refunded" and its stock and totals say "sold". The window is usually a few minutes, and month-end is when somebody finds the one that lasted a year.

18.4 Customers, and What You Are Keeping

The customer side of the back office is shorter but carries more risk, because the records are about people.

What Customers › Customers lets you do, and the question attached to each

- **Edit a customer's details.** Who is allowed to, and is the change recorded? An order copies its addresses (Chapter 10), so editing the customer does not rewrite history — which is correct, and worth confirming rather than assuming.
- **Reset a password.** The shop cannot show you the old one, and anyone who offers to has stored it wrongly (Chapter 15).
- **See everything they bought.** Convenient for support, and a complete purchase history of a named person — which Chapter 22 treats as the regulated asset it is.
- **Add store credit** (`oc_customer_transaction`). This is real money the shop owes, and it is the one place in the back office where an administrator can create a liability with no approval step.
- **Delete a customer.** Their orders remain, because the orders are the shop's accounting records and are not the customer's to delete. Be able to explain that distinction before you are asked it.

Common Pitfalls

- **Assuming the admin panel writes to the database.** It calls the storefront, and can be refused.
- **Leaving `oc_api_ip` empty** and then reporting that order editing is broken without knowing why.
- **Reading the status names to predict stock.** Refunded returns stock and Reversed does not.
- **Putting a money-returned status into `config_processing_status`**, which is how Reversed got there.
- **Believing a return marked Refunded refunded anything.**
- **Recording the return before moving the order**, leaving a window where the two disagree.
- **Refunding without reversing commission**, which Chapter 12 named as the commonest unreconciled figure in a small shop.

- **Editing an order to "correct" it** instead of changing its status, which loses the history a dispute would be settled from.

Try It Yourself: Refund an order properly, then find what the shop still believes

```
# --- 1. Find out why order editing does not work --------------------
cd code
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT * FROM oc_api; SELECT * FROM oc_api_ip;"
# One table has a row and the other does not. Say what that means.

# --- 2. Prove it from outside ---------------------------------------
python m18_orders.py --api

# --- 3. Predict the stock table, then watch it ----------------------
# Which of these returns goods to stock: Shipped, Canceled, Refunded,
# Reversed? Commit to an answer for each BEFORE running:
python m18_orders.py --stock

# --- 4. Explain the two that surprised you --------------------------
docker compose exec db mariadb -ushop -pshoppw opencart \
  -e "SELECT value FROM oc_setting \
      WHERE \`key\` IN ('config_processing_status', \
                        'config_complete_status');"

# --- 5. Record a return in the admin panel --------------------------
# Admin: Sales > Returns > Add New. Action: Refunded. Save.
# Now check: did the order's status change? Did stock move? Did any
# money move? Check all three in the database, not on the screen.
python m18_orders.py --refund

# --- 6. Do the refund properly --------------------------------------
# Follow the five-step procedure in this chapter. Write down which
# steps the software performed and which you performed.

# --- 7. Find the disagreement ---------------------------------------
# Admin: Reports > Sales > Orders. Does the total match what the shop
# actually kept? Account for every difference.

# --- 8. Write the procedure -----------------------------------------
# One page a shop assistant could follow, with the order of steps
# and a stated reason for that order.

# --- 9. Clean up ----------------------------------------------------
python m18_orders.py --clean
```

What to notice. Step 5 is the one students argue about. The screen says **Refunded**, the save succeeds, a confirmation appears — and the database shows two rows written in a table nothing else reads.

Checkpoint — Test Yourself

1. An order is moved from Pending to Shipped, then to Reversed. State what happened to stock at each step and why, and say what you would change in the shop's configuration if you wanted Reversed to behave like Refunded.
2. A shop records 40 returns a month, all marked **Refunded**, and its stock figures drift steadily away from reality. Explain the mechanism precisely, say what else is also drifting, and give the smallest change that would stop it.

3. The admin panel reports that it cannot change an order. Describe how you would diagnose this in four steps, naming what you would look at each time, and explain why the fault is easy to misread as a permissions problem with the administrator's account.

Chapter Summary

- The admin panel changes an order by signing a request and calling the storefront's API, so that stock, notification and history follow one code path whatever triggered the change.
- Measured: the API user ships enabled with an *empty* address allow-list, which permits nothing — so every signed call is refused, including the admin panel's own. The third inert-feature fault in this course.
- Stock is decided by the transition between sets of statuses, not by the status. Moving into the processing-or-complete set subtracts; moving out of it adds back; moving within it does nothing.
- Measured: Shipped moved no stock. Canceled and Refunded returned it. **Reversed did not**, because 12 sits in `config_processing_status` — a setting, and therefore your responsibility.
- A return marked **Refunded** performs two SQL statements and nothing else: no order status change, no stock, no credit, no money. It is a note.
- A refund is five jobs and the software does one. The other four need a written procedure, in a stated order, because the order of steps decides how long the records disagree.
- Store credit is a liability an administrator can create with no approval step. Deleting a customer does not delete their orders, and should not.

Review Questions

1. **(MCQ)** Moving an order to status 12 (Reversed) causes stock to: (a) be returned; (b) be subtracted again; (c) stay as it is; (d) depend on the payment method.

2. **(MCQ)** Recording a return with the action **Refunded**: (a) refunds the customer; (b) restores stock; (c) changes the order's status; (d) writes two rows and does nothing else.

3. **(Short)** Explain why the admin panel calls the storefront's API instead of writing to the database, and give one problem that design prevents.

4. **(Short)** Why does an empty `oc_api_ip` refuse every call rather than allowing every call? Give the general lesson about allow-lists.

5. **(Short)** An order's addresses are copied at checkout. Explain what this means for editing a customer's address afterwards, and why it is the correct behaviour.

6. **(Applied)** A shop's stock counts have drifted by several hundred units over six months. Give four distinct mechanisms from this chapter that could produce that, say how you would test for each in the database, and state which you would expect to find first.

7. **(Applied)** Write the refund procedure for a shop taking 200 orders a day with a 6% return rate, two staff in the back office and a payment provider whose console is separate from the shop. State each step, who performs it, what is recorded, the order and the reason for the order, and the daily check that would catch a missed step.